

TME 1 - Virtualisation complète et réseaux virtuels

Contents

Définitions	2
Architecture generale du TME	4
1 Virtualisation complete	5
1.1 Question 1 — Support de la virtualisation	5
1.2 Question 2 — Nombre de processeurs reels	5
1.3 Question 3 — Drapeaux de virtualisation	5
1.4 Question 4 — Analyse depuis la VM Debian	5
2 Creation de VM QEMU/KVM avec virt-manager	6
2.1 Question 1 — Controleur IDE dans pciconf	7
2.2 Question 2 — Interface PCI de la carte reseau	7
2.3 Question 3 — Ressources detectees par FreeBSD	7
2.4 Question 4 — Adresse MAC	7
2.5 Question 5 — Ping vers la VM Linux	7
2.6 Question 6 — Ping vers la machine réelle	8
3 Connectivite de la VM et reseau virtuel	8
3.1 Question 1 — Caracteristiques du reseau virtuel	8
3.2 Question 2 — Reseau de la VM FreeBSD	8
3.3 Question 3 — Autres modes de connexion	9
3.4 Question 4 — Nature du reseau default	9
3.5 Question 5 — NAT et translation d’adresses	9
3.6 Question 6 — Connexion bridge ↔ VM FreeBSD	10
3.7 Question 7 — Service DHCP	11
4 Reseaux de machines virtuelles	11
4.1 Question 1 — Peripherique du reseau testres	11
4.2 Question 2 — Format qcow2 et taille des disques	12
4.3 Question 3 — Proprietes des VMs clonees	12
4.4 Question 4 — Adresses MAC/IP des clones	12
4.5 Question 5 — Interfaces TAP des clones	12
4.6 Question 6 — Conflit d’adresses reseau	13
4.7 Complement : MACVTAP (section IV.8 du TME)	13

Définitions

Bus PCI (Peripheral Component Interconnect) Bus standard d'interconnexion des peripheriques. Dans une VM, les peripheriques emules par QEMU apparaissent sur un bus PCI virtuel. `pciconf -l` (FreeBSD) ou `lspci` (Linux) liste les peripheriques PCI detectes avec leur adresse bus:slot:fonction. Le PCI est un standard de connexion informatique qui permet de relier des composants matériels à la carte mère d'un ordinateur. Il agit comme un canal de communication principal entre le processeur et divers périphériques internes tels que les cartes graphiques, les cartes réseau ou les cartes son. Aujourd'hui, on utilise principalement sa version évoluée appelée PCI Express qui offre des débits beaucoup plus élevés. Ce système est essentiel pour personnaliser ou améliorer les capacités d'un ordinateur car il permet d'ajouter ou de remplacer facilement des composants selon les besoins de l'utilisateur..

dnsmasq Serveur léger combinant DNS et DHCP. Utilisé par libvirt pour fournir les services réseau aux VMs. Écoute sur l'adresse du bridge (ex: 192.168.122.1) et configure via `/var/lib/libvirt/dnsmasq/..`

Hyperviseur Logiciel qui crée et gère des VMs. Type 1 (bare-metal) : directement sur le matériel (ex: VMware ESXi, Xen). Type 2 (hosted) : au-dessus d'un OS hôte (ex: VirtualBox). KVM est un cas hybride : module noyau Linux transformant Linux en hyperviseur de type 1..

IDE / ATA (Integrated Drive Electronics) Interface disque ancienne gérant jusqu'à 4 périphériques : primary master (hda/ad0), primary slave (hdb/ad1), secondary master (hdc/ad2), secondary slave (hdd/ad3). Dans le TME, le disque FreeBSD doit être en hdd (secondary slave) pour correspondre à sa configuration d'origine (`/dev/ad3`)..

KVM (Kernel-based Virtual Machine) Module du noyau Linux transformant le noyau en hyperviseur de type 1. Exploite VT-x/AMD-V pour exécuter des VMs avec des performances proches du natif. Utilisé conjointement avec QEMU pour l'émulation des périphériques..

macvtap Interface réseau virtuelle Linux combinant macvlan et TUN/TAP. Plusieurs interfaces macvtap peuvent être attachées à un même NIC physique. Modes : Pont/VEB (bridge virtuel), VEPA (tout passe par le réseau externe), Privé (isolation totale), Passthrough (un seul invité, accès direct). Inconvénient : l'hôte ne peut pas communiquer avec les VMs en mode Pont..

NAT (Network Address Translation) Technique réseau traduisant des adresses IP privées en adresse publique. Le mode MASQUERADE (iptables) remplace l'IP source par l'IP de l'interface de sortie. Les réponses sont re-mappées grâce à une table de connexions (`nf_conntrack`). Utilisé par libvirt pour donner aux VMs accès à Internet..

qcow2 (QEMU Copy-On-Write v2) Format d'image disque QEMU. Supporte le Copy-on-Write : un disque overlay ne stocke que les blocs modifies par rapport au backing file. La taille reelle sur disque est bien inferieure a la taille virtuelle declaree. Tres utile en cloud pour creer massivement des VMs legeres depuis une image de base..

QEMU Emulateur et virtualiseur open-source simulant differentes architectures materielles. Associe a KVM, QEMU gere l'emulation des peripheriques (carte reseau, disque...) pendant que KVM assure l'execution CPU acceleree. QEMU seul peut fonctionner en pure emulation logicielle..

TUN/TAP Interfaces reseau virtuelles du noyau Linux. TAP (couche 2) emule une interface Ethernet ; mode utilise par QEMU pour connecter les VMs au bridge. TUN (couche 3) travaille au niveau IP ; utilise pour les VPNs. QEMU cree automatiquement une interface TAP (`vnet0`, `vnet1...`) par VM demarree..

vCPU (Virtual CPU) Processeur virtuel presente a une VM. Mappe sur un ou plusieurs CPUs physiques. Les vCPUs d'une VM ne voient generalement pas les flags `vmx/svm` sauf si la virtualisation imbriquee est explicitement activee..

Virtualisation complete Mode ou le systeme invite s'execute sans modification. L'hyperviseur emule entierement le materiel. Necessite les extensions materielles VT-x (Intel) ou AMD-V (AMD). Le systeme invite n'a pas conscience d'etre virtualise. Oppose a la para-virtualisation qui necessite un OS modifie..

Virtualisation imbriquee (Nested Virtualization) Capacite d'executer un hyperviseur dans une VM. Necessite que l'hyperviseur hote expose les flags `vmx/svm` aux vCPUs. Dans le TME, les vCPUs de la VM Debian n'ont pas ces flags : les VMs QEMU/KVM fonctionnent donc en emulation logicielle pure, ce qui est moins performant mais fonctionnel..

VT-x / AMD-V Extensions materielles de virtualisation. VT-x (Intel) et AMD-V (AMD) ajoutent un nouveau mode d'execution (VMX root/non-root) permettant a l'hyperviseur de prendre le controle. Le flag `vmx` (`/proc/cpuinfo`) indique VT-x ; `svm` indique AMD-V..

Architecture generale du TME

Architecture a trois niveaux imbriques

Le TME met en oeuvre une architecture de virtualisation imbriquee :

1. **Machine reelle PPTI** : serveur physique Linux, 12 CPUs, support VT-x actif.
2. **VM VirtualBox (systeme hote)** : VM Debian lancee via vbox CRV4. C'est l'hote pour les VMs QEMU/KVM.
3. **VMs QEMU/KVM (systemes invites)** : VMs FreeBSD creees avec virt-manager dans la VM Debian.

Les VMs invitees ne voient pas les flags VT-x ([Virtualisation imbriquee \(Nested Virtualization\)](#) non activee), mais [QEMU/KVM \(Kernel-based Virtual Machine\)](#) reste fonctionnel en emulation logicielle.

Commandes essentielles du TME

Commande	Usage
<code>lscpu</code>	Infos CPU (nombre, architecture, flags)
<code>cat /proc/cpuinfo</code>	Flags CPU detailles (vmx, svm, hypervisor...)
<code>pciconf -l</code>	Liste peripheriques PCI (FreeBSD)
<code>sysctl hw.model hw.ncpu hw.realmem</code>	Infos materiel FreeBSD
<code>ip link show type bridge</code>	Bridges reseau actifs
<code>brctl show</code>	Details bridges et ports attaches
<code>ip link show type tun up</code>	Interfaces TUN/TAP actives
<code>ip tuntap</code>	Mode (tun/tap) des interfaces virtuelles
<code>iptables -t nat -L -n -v</code>	Regles NAT iptables
<code>cat /proc/net/nf_conntrack</code>	Table de suivi des connexions noyau
<code>netstat-nat -nN</code>	Connexions NAT actives (lisible)
<code>qemu-img info fichier.qcow2</code>	Infos image disque (taille reelle vs virtuelle)
<code>qemu-img create -b base -f qcow2 out.qcow2</code>	Creation overlay qcow2
<code>virt-xml vm --update --add-device --disk ...</code>	Modification disque VM en CLI
<code>dmidecode -s system-product-name</code>	Modele de la machine (reelle ou virtuelle)

1. Virtualisation complete

Notion : Virtualisation complete et support materiel

La **Virtualisation complete** repose sur les extensions **VT-x / AMD-V**. L'**Hyperviseur KVM (Kernel-based Virtual Machine)** exploite ces extensions pour executer les instructions privilegiees des VMs directement sur le CPU physique. Le flag **vmx** (Intel) ou **svm** (AMD) dans `/proc/cpuinfo` confirme la disponibilite du support materiel. Sans ces flags, la virtualisation reste possible en emulation logicielle pure (plus lente).

1.1. Question 1 — Support de la virtualisation

La machine peut-elle effectuer de la virtualisation complete ?

Oui, parce qu'on a le flag **vmx** est present dans `/proc/cpuinfo` sur la machine reelle sous architecture **x86_64**, ce qui confirme la prise en charge materielle de la virtualisation complete. On peut également voir dans `lscpu` qu'on est en Virtualisation VT-x.

1.2. Question 2 — Nombre de processeurs reels

Combien y a-t-il de processeurs reels (CPU) ?

Il y a **12 processeurs reels (CPU)**, visible a la ligne **Processeur(s)** de `lscpu`.

1.3. Question 3 — Drapeaux de virtualisation

Les drapeaux (flags) des processeurs montrant le support de virtualisation sont-ils presents ?

Oui, le drapeau **vmx** (Intel VT-x) est present dans la sortie de `cat /proc/cpuinfo`, confirme la prise en charge materielle.

1.4. Question 4 — Analyse depuis la VM Debian

Notion : Flags CPU dans une VM

Les **vCPU (Virtual CPU)s** d'une VM ne voient pas forcément **vmx/svm** (sauf si la **Virtualisation imbriquee (Nested Virtualization)** est activee). Le flag **hypervisor** est toujours present pour signaler l'execution dans une VM. Le champ **hypervisor vendor id** indique l'interface de l'hyperviseur (**KVM, VMwareVMware, Microsoft Hv...**).

4a. Quel est le type de virtualisation de la VM Debian ?

Virtualisation complete via **KVM (Kernel-based Virtual Machine)**. QEMU emule les peripheriques, KVM assure l'acceleration CPU.

4b. Combien de vCPU dans la VM ?

La VM contient **6 vCPU (Virtual CPU)s**.

4c. Les vCPUs supportent-ils la virtualisation imbriquee ?

Non. Les flags **VT-x/vmx** sont absents des vCPUs. La **Virtualisation imbriquee (Nested Virtualization)** n'est pas activee. Les VMs QEMU/KVM fonctionneront en emulation logicielle pure.

4d. Quel flag distingue les vCPUs geres par un hyperviseur ?

Le flag **hypervisor**, positionne par l'hyperviseur pour signaler a l'OS invite qu'il s'execute dans un environnement virtualise.

4e. Quel est le constructeur/interface de l'hyperviseur ?

KVM (Kernel-based Virtual Machine), visible via **hypervisor vendor id** dans **/proc/cpuinfo**.

4f. Quelle est la machine et son fabricant ?

La machine est **VirtualBox**, fabricant **innotekGmbH (Oracle)**, obtenu avec **dmidecode -s system-product-name** et **dmidecode -s system-manufacturer**.

2. Creation de VM QEMU/KVM avec virt-manager

Notion : Controleur IDE et nommage des disques

Un controleur **IDE / ATA (Integrated Drive Electronics)** gere 4 peripheriques. Sous Linux/QEMU : **hda** (primary master), **hdb** (primary slave), **hdc** (secondary master), **hdd** (secondary slave). Sous FreeBSD : **ad0**, **ad1**, **ad2**, **ad3**. Le disque FreeBSD du TME est configure pour **/dev/ad3s1a** (secondary slave), donc il faut le connecter en **hdd**. La configuration est visible dans **/etc/libvirt/qemu/freebsd.xml** dans l'element **<disk><target dev="hdd">**.

Notion : Fichier XML libvirt et virt-xml

Chaque VM est decrite par un fichier XML dans **/etc/libvirt/qemu/**. On peut le modifier via **virt-manager (GUI)** ou en CLI avec **virt-xml** :

```
virt-xml freebsd -update -remove-device -disk target=hda
virt-xml freebsd -update -add-device -disk path=/chemin/disque,target=hdd
```

2.1. Question 1 — Controleur IDE dans pciconf

Quelle ligne de `pciconf -l` correspond a `<controller type='ide'>` dans `freebsd.xml` ?

```
atapci@pci0:0:1:1
```

`atapci` designe le controleur ATA/IDE emule par QEMU sur le bus [Bus PCI \(Peripheral Component Interconnect\)](#) virtuel.

2.2. Question 2 — Interface PCI de la carte reseau

Quelle interface PCI connecte la carte reseau virtuelle ?

L'interface est `em0@pci0:0:3:0`. Dans `freebsd.xml` :

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x03'
function='0x0' />
```

2.3. Question 3 — Ressources detectees par FreeBSD

Combien de CPUs dans le système FreeBSD? Modele ? RAM ?

FreeBSD a detecte **2 CPUs** (`cpu0`, `cpu1`), modele **QEMU Virtual CPU version 2.5+**, fabricant **AMD**. La RAM est **2048 MB** (verifiable avec `sysctl hw.realmem`).

2.4. Question 4 — Adresse MAC

L'adresse MAC est-elle identique entre la VM et virt-manager ? En utilisant la commande `ifconfig`

Oui. L'adresse MAC `52:54:00:a7:c1:f7` est identique dans la VM (`ifconfig`) et dans virt-manager. Le prefixe `52:54:00` est la plage reservee par QEMU.

2.5. Question 5 — Ping vers la VM Linux

Le ping vers la VM Linux fonctionne-t-il ?

Oui. Le ping marche entre la VM FreeBSD à l'adresse IP de la carte réseau de la VM Linux via le reseau virtuel `default` (bridge `virbr0`).

2.6. Question 6 — Ping vers la machine réelle

Le ping vers la machine réelle (10.4.0.xx) fonctionne-t-il depuis VM FreeBSD?

Oui. Le chemin des paquets est : VM FreeBSD → interface TAP → bridge **virbr0** → **NAT** (Network Address Translation) MASQUERADE sur **enp0s3** → réseau PPTI → machine réelle. Les autres VMs servent d'intermediaires via routage et NAT.

3. Connectivite de la VM et reseau virtuel

Notion : Architecture complete d'un reseau virtuel libvirt

Quand libvirt cree le reseau **default**, il met en place automatiquement :

1. Un **bridge Linux** **virbr0** avec l'IP 192.168.122.1 (passerelle des VMs)
2. Un processus **dnsmasq** sur le bridge pour le DHCP (.2 a .254) et le DNS
3. Des **regles iptables NAT** (MASQUERADE) pour l'accès Internet des VMs
4. Pour chaque VM demarree : une **interface TAP** (vnet0...) ajoutée comme port du bridge

Ce mecanisme s'observe avec `brctl show` (bridge + ports) et `iptables -t nat -L -n -v` (regles NAT).

3.1. Question 1 — Caracteristiques du reseau virtuel

Quelles sont les caracteristiques du réseau virtuel crée ?

- **Nom** : default
- **Peripherique associe** : virbr0 (bridge Linux)
- **Adresse reseau** : 192.168.122.0/24
- **Attribution des IPs** : Les systèmes invités connectés au réseau virtuel peuvent obtenir les configurations IP grâce au DHCP, le DHCP permet de donner une adresse IP à une machine qui vient de se connecter au réseau.
- **Acces Internet** : Les VMs sont connectées aux réseaux externes (Internet) grâce au protocole NAT. Le NAT convertit une adresse IP privé en adresse IP public, car en IPv4 on peut ne pas avoir assez d'adresses IP privées donc, grâce au NAT, nous n'avons plus ce problème.

3.2. Question 2 — Reseau de la VM FreeBSD

A quel reseau la VM FreeBSD est-elle connectee ?

Au reseau **default**, via le bridge **virbr0**.

3.3. Question 3 — Autres modes de connexion

Quels autres modes de connexion sont disponibles ?

- Se connecter aux périphériques de l'hôte `enp0s3` : `macvtap` (modes : Pont/VEB, VEPA, Prive, Passthrough)
- **Péripherique partage** : nom de peripherique reseau specifie manuellement

3.4. Question 4 — Nature du reseau default

Qu'est-ce que le reseau virtuel default en realite ?

C'est un **bridge Linux** (`virbr0`), confirme par `brctl show` qui liste `virbr0` comme bridge avec ses ports TAP attaches.

3.5. Question 5 — NAT et translation d'adresses

Notion : Suivi des connexions NAT

`/proc/net/nf_conntrack` maintient l'état de toutes les connexions suivies par le noyau Netfilter. Pour chaque connexion, on voit l'IP source originale (VM), l'IP de destination, et l'IP de translation (après MASQUERADE). `netstat-nat -nN` affiche ces informations de façon plus lisible sous forme de tableau Proto/NATed/ NAT-host/ Destination.

```
root@debian:~# cat /proc/net/nf_conntrack
ipv4      2  udp      17 13 src=10.4.0.83 dst=224.0.0.251 sport=5353 dport=5353 [UNREPLIED] src=224.0.0.251 dst=10.4.0.83 sport=5353 dport=5353 mark=0 zone=0 use=2
ipv4      2  icmp      1 29 src=192.168.122.98 dst=10.4.0.3 type=8 code=0 id=31495 src=10.4.0.3 dst=10.4.0.119 type=0 code=0 id=31495 mark=0 zone=0 use=2
ipv6      10  udp      17 13 src=fe80:0000:0000:0000:0000:0000:00ff:fe04:0102 dst=ff02::1:3 sport=5353 dport=5353 [UNREPLIED] src=ff02::1:3 sport=5353 dport=5353 mark=0 zone=0 use=2
ipv4      2  udp      17 13 src=10.4.0.104 dst=224.0.0.251 sport=5353 dport=5353 [UNREPLIED] src=224.0.0.251 dst=10.4.0.104 sport=5353 dport=5353 mark=0 zone=0 use=2
```

Figure 1: `cat /proc/net/nf_conntrack`

```
root@debian:~# netstat-nat -nN
Proto NATed Address      NAT-host Address      Destination
Address      State
icmp 192.168.122.98      10.4.0.119            10.4.0.3
```

Figure 2: `netstat-nat -nN`

5.1. Quelle adresse IP est utilisee pour la translation ?

L'adresse IP de la VM FreeBSD dans le reseau virtuel : 192.168.122.98.

5.2. Sur quelle interface la translation est-elle effectuee ?

Sur l'interface `enp0s3` (10.4.0.119), interface de sortie de la VM Debian vers le reseau PPTI.

3.6. Question 6 — Connexion bridge ↔ VM FreeBSD

Notion : Role de l'interface TAP dans QEMU

QEMU cree une interface **TUN/TAP** (mode TAP) pour chaque VM demarree. Cette interface est un peripherique reseau virtuel de couche 2 : les trames Ethernet de la carte emule de la VM arrivent dans le noyau hote via l'interface TAP. Le bridge les transmet vers la bonne destination. QEMU fait le lien entre l'interface TAP (cote noyau hote) et la carte reseau emule (cote VM). Aucun cable physique n'est implique : tout est logiciel dans le noyau.

Comment la VM FreeBSD est-elle connectee au bridge virbr0 ?

Via une interface **TAP** (ex: `vnet0`), creee par QEMU au demarrage et ajoutee comme port du bridge `virbr0`. Commande : `ip link show up | grep master` et `ip link show type tun up`

6.1. Pourquoi pas d'adresse IP sur l'interface TAP ?

L'interface TAP opere en couche 2 (trames Ethernet brutes). Elle n'a pas besoin d'IP : c'est le bridge (`virbr0` : 192.168.122.1) qui a une IP (passerelle), et la VM FreeBSD a sa propre IP via DHCP. L'interface TAP est un "cable virtuel" entre le bridge et la VM.

6.2. Quel est le type de cette interface ?

TUN/TAP — mode **TAP** (couche 2, trames Ethernet completes), par opposition au mode TUN (couche 3, paquets IP seuls).

6.3. Destination finale des donnees du bridge ?

La **VM FreeBSD**. **QEMU** intercepte les donnees de l'interface TAP et les transmet a la carte reseau emule (`em0`) presentee a l'OS FreeBSD.

6.4. Mode de l'interface ?

Mode **TAP**, verifiable avec `ip tuntap | grep vnet0`.

3.7. Question 7 — Service DHCP

7.1. Quel systeme joue le role du serveur DHCP ?

La VM hote **Linux Debian** (le systeme qui fait tourner QEMU/KVM).

7.2. Quel processus implemente le DHCP ?

dnsmasq, qui assure a la fois DHCP et DNS pour les VMs.

7.3. Fichier de configuration et intervalle d'adresses ?

Fichier : `default.conf` (dans `/var/lib/libvirt/dnsmasq/`). Plage DHCP : 192.168.122.2 a 192.168.122.254.

7.4. Adresse du serveur DNS ? Regardez dans le fichier `etc/resolv.conf` de la VM `freebsd`

192.168.122.1 — l'adresse du bridge `virbr0` lui-meme, sur lequel **dnsmasq** ecoute les requetes DNS des VMs.

4. Reseaux de machines virtuelles

Notion : qcow2 et Copy-on-Write en detail

La commande `qemu-img create -b freebsd.vmdk -f qcow2 freebsd1.qcow2` cree un fichier overlay qui reference le vmdk comme *backing file*. Initialement, l'overlay est quasiment vide (quelques Mo de metadonnees). Au premier acces en **lecture**, les donnees viennent du backing file. Au premier acces en **ecriture** d'un bloc, ce bloc est copie dans l'overlay, puis modifie (copy-on-write). La taille reelle de l'overlay n'augmente qu'en cas d'ecriture.

Fichier	Taille reelle	Role
<code>freebsd.vmdk</code>	7,1 Go	Backing file (image de base complete)
<code>freebsd1.qcow2</code>	≈12 Mo	Overlay (differences uniquement)
<code>freebsd1.qcow2</code> (apres dd 10M)	≈23 Mo	Overlay avec 10 Mo ecrits

4.1. Question 1 — Peripherique du reseau testres

Quel peripherique est associe au reseau testres ?

`virbr1`, second bridge cree par libvirt pour le reseau 172.27.27.0/24.

4.2. Question 2 — Format qcow2 et taille des disques

2.1. Taille globale du disque et interet du qcow2 ?

La taille virtuelle vue par la VM (`df -h`) est ≈ 121 Go. Le fichier `freebsd1.qcow2` ne pese que **12 Mo** sur le disque hôte. Le format **qcow2** (QEMU Copy-On-Write v2) permet de creer massivement des VMs legeres sans dupliquer l'image de base.

2.2. Coherence entre `ls -lh` et `qemu-img info` ?

Oui. Les deux affichent la meme taille réelle sur disque. `qemu-img info` distingue aussi la *taille virtuelle* (declaree, vue par la VM) de la *disk size* (réelle, occupee sur le disque hôte).

2.3. Taille apres ecriture de 10 Mo dans la VM ?

La taille de `freebsd1.qcow2` passe de **12 Mo a 23 Mo** : augmentation de ≈ 10 Mo, conformement au principe copy-on-write du format qcow2.

4.3. Question 3 — Proprietes des VMs clonees

Les 3 VMs clonees ont-elles les memes proprietes ?

Oui : meme nombre de CPUs, meme RAM, meme souris/clavier. Seuls l'UUID, le nom, l'adresse MAC sont différent.

4.4. Question 4 — Adresses MAC/IP des clones

Adresses MAC/IP des 3 clones et connectivite ?

VM	Adresse MAC	Adresse IP
freebsd1	52:54:00:f0:cb:79	172.27.27.6
freebsd2	52:54:00:c9:47:60	172.27.27.8
freebsd3	52:54:00:06:85:e6	172.27.27.5

Les trois clones sont interconnectes via le bridge `virbr1` (reseau `testres`). Les pings fonctionnent dans les deux sens.

4.5. Question 5 — Interfaces TAP des clones

Quelles interfaces TAP sont associees a chaque clone ?

- `freebsd1` : `vnet1`
- `freebsd2` : `vnet2`
- `freebsd3` : `vnet0`

Ces interfaces TAP sont creees par QEMU au demarrage et ajoutes comme ports de virbr1.

4.6. Question 6 — Conflit d'adresses reseau

Que se passe-t-il en creant deux reseaux avec la meme adresse ?

```
Erreur : internal error : Network is already in use by interface
virbr1.
```

Impossible de creer deux reseaux virtuels avec le meme espace d'adressage : cela creerait un conflit de routage dans la table de routage de l'hote Linux.

4.7. Complement : MACVTAP (section IV.8 du TME)

Notion : Modes MACVTAP et leurs usages

L'interface `macvtap` connecte les VMs directement a un NIC physique sans bridge intermediaire. Les VMs recoivent une IP sur le meme reseau que l'interface physique (`enp0s3`). **Inconvenient majeur** : en mode Pont, l'hote ne peut pas communiquer avec les VMs (la `macvtap` bypass la pile reseau de l'hote pour les echanges hote↔VM).

Mode	Description
Pont (VEB)	VMs communiquent entre elles + acces externe. L'hote ne peut pas pinger les VMs.
VEPA	Tout passe par le reseau externe, meme entre VMs. Necessite switch en mode hairpin (IEEE 802.1Qbg).
Prive	Comme VEPA, mais isolation totale entre VMs meme avec hairpin.
Passthrough	Une seule VM recoit le NIC directement. Meilleures performances.